Abstract

Plant diseases can reduce crop quality and yield. Identifying symptoms early helps farmers take suitable action, but visual inspection may require experience and can be time-consuming. This mini project proposes a Convolutional Neural Network (CNN) that classifies images of plant leaves into disease categories using their visual patterns. The workflow includes collecting a labelled image dataset, preparing and splitting the images, training a CNN, and evaluating its predictions using accuracy, precision, recall, F1-score, and a confusion matrix. A pretrained CNN can optionally be fine-tuned to compare transfer learning with a model trained from scratch. The project is intended as an educational prototype; its predictions should not replace advice from an agricultural expert.

Keywords: Deep Learning, CNN, Image Classification, Plant Disease, Transfer Learning.

1. Introduction

Plant leaves often show visible symptoms such as spots, colour changes, or damaged areas when a plant is affected by disease. A computer vision model can learn visual patterns from labelled leaf images and predict the class that best matches a new image.

A Convolutional Neural Network is suitable for image tasks because convolutional layers learn local visual features, pooling layers reduce spatial dimensions, and classification layers use the learned features to produce class scores.

This project applies the CNN concepts covered in Unit III to a practical image-classification problem.

2. Problem Statement

Manual plant-leaf inspection can be slow and may be difficult for people who do not have specialised knowledge. The project explores whether a CNN can classify labelled leaf images into a defined set of healthy and diseased categories.

The prototype is limited to the categories represented in its training dataset. It cannot reliably diagnose every plant disease or guarantee correct predictions for images captured in different real-world conditions.

3. Objectives

1. Understand the basic operations and architecture of a CNN.
2. Prepare a labelled dataset of plant-leaf images.
3. Train an image-classification model.
4. Evaluate the model using suitable classification metrics.
5. Optionally compare a basic CNN with transfer learning.
6. Document the method, findings, limitations, and future improvements.

4. Scope

The project focuses on image classification for a selected subset of plant-leaf classes. It is designed as a small academic prototype that can be run in a Python environment with a deep-learning framework such as TensorFlow/Keras or PyTorch.

The project does not provide treatment recommendations, identify diseases outside its trained classes, or claim field-level reliability without further testing.

5. Dataset

A suitable starting point is the publicly available **PlantVillage** plant-leaf image dataset. The student should obtain it from an authorised source and follow the dataset's licence and usage conditions.

For a manageable first experiment, select a small number of classes—for example, healthy leaves and two or three disease categories for one plant species. Record the exact dataset source, selected classes, number of images per class, and any exclusions after downloading and inspecting the data.

Data preparation

=>Remove unreadable or duplicate images where feasible.

=>Resize images to a consistent input size, such as 128 × 128 pixels.

=>Normalize pixel values to a consistent numerical range.

=>Split the data into training, validation, and test sets.

=>Keep the test set separate until final evaluation.

=>Apply augmentation, such as small rotations or horizontal flips, only to training images where appropriate.

=>Check class balance and avoid allowing near-duplicate images to leak across splits.

Dataset details to complete after preparation

Item	**Actual value**
Dataset source and version	[Fill in]
Classes selected	[Fill in]
Total images used	[Fill in]
Training / validation / test split	[Fill in]
Image dimensions	[Fill in]

6. Proposed Methodology

The project follows these stages:

1. **Data collection:** Obtain labelled plant-leaf images.
2. **Preprocessing:** Resize, normalize, inspect, and split the images.
3. **Model design:** Build a CNN with convolution, activation, pooling, and classification layers.
4. **Training:** Train the model on the training set and monitor validation performance.
5. **Model selection:** Save the model that performs best on validation data, rather than choosing it based on test performance.

6. **Evaluation:** Evaluate the selected model once on the held-out test set.
7. **Analysis:** Review incorrect predictions and discuss limitations.

Workflow

Leaf images → Preprocessing → Train / Validation / Test split → CNN training → Evaluation → Predicted class

7. CNN Architecture

A basic CNN can use the following conceptual architecture:

Layer	Purpose
Input layer	Accepts a resized RGB leaf image
Convolution + ReLU	Learns visual features such as edges, textures, and spots
Max pooling	Reduces spatial dimensions
Second convolution + ReLU	Learns more complex visual patterns
Max pooling	Further reduces spatial dimensions
Flatten or global average pooling	Converts feature maps into a representation for classification
Dense layer	Combines learned features
Softmax output	Produces a probability distribution over the selected classes

The number of output units must match the number of selected classes. The model configuration, input dimensions, batch size, optimizer, learning rate, and number of epochs should be recorded after implementation.

8. Training and Hyperparameters

The model can be trained using categorical cross-entropy for one-hot encoded labels or sparse categorical cross-entropy for integer labels. An optimizer such as Adam can update the network weights through backpropagation.

Hyperparameters to document include:

=>Input image size

=>Batch size

=>Number of epochs

=>Learning rate

=>Optimizer

=>Loss function

=>Data augmentation settings

=>Early-stopping patience

Early stopping can be used when validation loss stops improving, helping reduce unnecessary training and overfitting. The test set must not be used to tune hyperparameters.

9. Evaluation Metrics

Accuracy alone may hide poor performance on under-represented classes. The project should report:

=>Accuracy: Proportion of all test predictions that are correct.

=>Precision: Among images predicted as a class, the proportion that truly belongs to that class.

=>Recall: Among images belonging to a class, the proportion correctly identified.

=>F1-score: Harmonic mean of precision and recall.

=>Confusion matrix: Counts of correct and incorrect predictions by class.

For a multi-class task, report per-class precision, recall, and F1-score, along with macro-average metrics where appropriate.

Results table — complete after running the experiment

Do not enter estimated or invented values. Replace the placeholders with actual test-set results.

Metric	**Measured result**
Test accuracy	[Fill in]
Macro precision	[Fill in]
Macro recall	[Fill in]
Macro F1-score	[Fill in]
Most commonly confused classes	[Fill in]

If transfer learning is implemented, include a separate row or table for each model and evaluate both using the same test split.

10. Expected Outcome

The expected outcome is a working prototype that learns visual patterns from the selected training images and predicts one of the chosen classes for a leaf image. The actual performance must be established experimentally and reported honestly. Performance on a curated dataset may be higher than performance on field photographs because lighting, backgrounds, camera quality, and disease severity can vary.

11. Applications

=>Educational demonstration of CNN image classification.

=>Preliminary screening support for plant-leaf images.

=>Demonstration of transfer learning and model evaluation.

=>A foundation for a future agricultural decision-support prototype.

The model should be treated as an experimental aid, not as a standalone diagnostic authority.

12. Limitations

1. Predictions are restricted to the classes represented in the training data.
2. Backgrounds and image conditions may differ from real agricultural settings.
3. Imbalanced data can lead to unequal class performance.
4. Similar-looking symptoms may be confused by the model.
5. Dataset bias or duplicate images can make evaluation misleading.
6. A correct-looking prediction does not guarantee a correct agricultural diagnosis.

13. Future Enhancements

=>Compare a CNN trained from scratch with a pretrained model such as MobileNetV2.

=>Test on independently collected field images.

=>Improve class balance and data diversity.

=>Add a simple interface for uploading a leaf image.

=>Explore model explainability methods to inspect which image regions influence predictions.

=>Measure inference time and model size for potential mobile or edge deployment.

14. Conclusion

This project applies deep convolutional neural networks to the task of plant-leaf image classification. It demonstrates a complete machine-learning workflow: dataset preparation, CNN design, training, validation, and evaluation. The project also highlights the importance of data quality, fair evaluation, and testing beyond curated images. Final conclusions about model performance should be written only after the experiment has been completed and measured results have been recorded.

References

1. Hughes, D. P., and Salathé, M. “An open access repository of images on plant health to enable the development of mobile disease diagnostics.” *arXiv preprint arXiv:1511.08060*, 2015.
2. LeCun, Y., Bengio, Y., and Hinton, G. “Deep learning.” *Nature*, 521, 436–444, 2015.
3. Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016.
4. TensorFlow documentation, “Image classification.” https://www.tensorflow.org/tutorials/images/classification
5. PlantVillage dataset information and applicable licence: [Insert the exact source URL and access date used].